

Sponsored Periphery Audit

OpenZeppelin Security - December 17, 2025

Summary

Type: Cross Chain

Timeline: From 2025-10-23 → To 2025-11-12

Languages: Solidity

Findings

Total issues: 36 (32 resolved)

Critical: 0 (0 resolved) · High: 4 (3 resolved) · Medium: 5 (5 resolved) · Low: 11 (9 resolved)

Notes & Additional Information

12 notes raised (11 resolved)

Client Reported Issues

4 issues reported (4 resolved)

Scope

OpenZeppelin performed a differential audit of the [UMAProtocol /across-contracts-private](#) repository at HEAD commit [ec9bd79](#) against BASE commit [271da3e](#). As part of a subsequent update to reduce the contract size, the scope was expanded to include a differential audit of the [UMAProtocol /across-contracts-private](#) repository at HEAD commit [ec9bd79](#) against BASE commit [271da3e](#).

This site uses cookies to offer you a better user experience. Information about the cookies we use can be found in our Privacy Policy under "Cookie Policy". You can reject all non-essential cookies by choosing "Decline" or set your own preference under "Cookies settings".

[Cookies settings](#)

Accept

Decline

added to the scope as well.

In scope were the following files:

contracts

- └─ SpokePool.sol
- └─ ZkSync_SpokePool.sol
- └─ chain-adapters
 - └─ OP_Adapter.sol
- └─ external/interfaces
 - └─ ICoreDepositWallet.sol
- └─ handlers
 - └─ HyperliquidDepositHandler.sol
 - └─ MulticallHandler.sol
 - └─ PermissionedMulticallHandler.sol
- └─ libraries
 - └─ AddressConverters.sol
 - └─ HyperCoreLib.sol
 - └─ SponsoredCCTPQuoteLib.sol
- └─ periphery
 - └─ mintburn
 - └─ ArbitraryEVMFlowExecutor.sol
 - └─ AuthorizedFundedFlow.sol
 - └─ BaseModuleHandler.sol
 - └─ Constants.sol
 - └─ HyperCoreFlowExecutor.sol
 - └─ HyperCoreFlowRoles.sol
 - └─ Structs.sol
 - └─ SwapHandler.sol
 - └─ sponsored-cctp
 - └─ SponsoredCCTPDstPeriphery.sol
 - └─ SponsoredCCTPSrcPeriphery.sol
 - └─ sponsored-oft
 - └─ ComposeMsgCodec.sol
 - └─ DstOFTHandler.sol
 - └─ QuoteSignLib.sol
 - └─ SponsoredOFTSrcPeriphery.sol
 - └─ Structs.sol

System Overview

The system under review provides a framework for sponsored, cross-chain transactions that originate from a source EVM-compatible chain and are fulfilled on the destination chain, HyperEVM. From HyperEVM, the system can then interact with the HyperCore L1 for final settlement. It leverages two underlying bridging protocols, Circle's Cross-Chain Transfer Protocol (CCTP) and LayerZero's Omnichain Fungible Token (OFT), to facilitate the transfer of assets. The architecture is designed to support both simple 1:1 asset transfers and more complex operations. The entire process is initiated by a user submitting a quote that has been authorized and signed by a trusted off-chain API that is managed by Across. This quote dictates the parameters of the transaction, including the final recipient, the assets involved, and the desired execution flow.

The user-facing entry points of the system are the source periphery contracts, `SponsoredCCTPSrcPeriphery.sol` and `SponsoredOFTSrcPeriphery.sol`, each tailored to a specific bridging protocol. These contracts are responsible for receiving a user's deposit along with the signed quote. Their main function is to validate the integrity of the quote by checking the signature, nonce, and deadline. Upon successful validation, they prepare the transaction payload and initiate the bridging process by calling the appropriate function on the underlying CCTP or OFT messenger contract.

On the destination chain, `SponsoredCCTPDstPeriphery.sol` and `DstOFTHandler.sol` act as the receivers for incoming messages from the bridges. Their role is to authorize the incoming message and route the transaction to the correct execution logic. The authorization model differs between the two: the CCTP contract re-validates the off-chain API's signature, while the OFT handler trusts messages only if they originate from a pre-configured, authorized source contract. After validation, these contracts decode the payload and delegate execution to the appropriate flow executor based on the `executionMode` specified in the original quote.

`HyperCoreFlowExecutor.sol` is a core component that orchestrates all interactions with the HyperCore L1. It is designed to only handle stablecoin-to-stablecoin flows, where the `baseToken` (for example USDC or USDT0) can be swapped for other dollar-pegged stablecoins. It manages sponsorship funds from a central `DonationBox` contract and supports a simple transfer flow for direct bridging, a two-stage asynchronous swap flow that relies on an off-chain permission bot for execution, and a fallback flow that settles funds on HyperEVM if any pre-conditions for a HyperCore transfer fail. This fallback mechanism is a critical safety feature that has been put in place to prevent the loss of user funds.

The `ArbitraryEVMFlowExecutor.sol` contract extends the system's capabilities by allowing for the execution of swaps on a Private Market Maker (PMM) or a DEX on HyperEVM before the final settlement. It acts as an intermediary that uses a `MulticallHandler` to execute a series of calls. The `actionData` for these calls is specified by the trusted off-chain API, not the end-user. After the arbitrary actions are complete, it passes the final token amounts to the `HyperCoreFlowExecutor` logic to be settled either on HyperCore or on HyperEVM.

The `HyperliquidDepositHandler` contract, which was later added to the scope, facilitates bridging of ERC20 tokens to end-user accounts on Hypercore. It serves as a dedicated handler for token deposits, supporting both direct interactions and those originating from the Across protocol. The contract manages a configurable list of supported tokens, including their respective activation fees, and incorporates a mechanism to activate new Hypercore user accounts, potentially utilizing a designated `DonationBox` for this purpose.

Security Model and Trust Assumptions

The system's security relies on a hybrid model that combines cryptographic signatures for authorization, the security of the underlying bridging protocols, and trust in several privileged roles and off-chain components. As such, the integrity of the entire process is contingent on the correct configuration of the contracts and the honest behavior of these trusted entities.

- **Off-Chain API and Signer:** The system's primary authorization mechanism is a quote signed by a trusted off-chain API that is managed by Across. The security of this signer's private key is critical. This entity is trusted to perform comprehensive input validation and to refuse to sign harmful quotes. Specifically, it is assumed to:
 - ensure that transaction amounts are non-zero
 - ensure that the deadlines are set to a reasonable duration
 - ensure that `finalToken` is a valid, non-zero address
 - ensure that `baseToken` and `finalToken` are different for arbitrary EVM flows
 - (for CCTP flows) correctly set the `destinationCaller` to the `SponsoredCCTPDstPeriphery` contract address and ensure that the `mintRecipient` is a valid EVM address to prevent irrecoverable loss of funds
 - provide safe `actionData` for arbitrary execution flows. This data is generated by the trusted API, not the user, and is assumed to be simulated and verified to perform as intended without malicious side effects
 - constrain `maxUserSlippageBps` to a reasonable range to prevent underflows and set a correct and safe `maxBpsToSponsor` value
 - verify that the `burnToken` in a CCTP quote is identical to the `baseToken` configured in the `HyperCoreFlowExecutor`, ensuring that the asset bridged to the destination chain matches the one expected by the core logic.
- **Economic Stability:** The system's economic model assumes that all tokens involved in swaps are stablecoins that maintain a tight 1:1 peg. It is not designed to handle the "black swan" de-peg event, which could lead to significant financial loss, particularly in sponsored flows where the system aims to provide a 1:1 value exchange.
- **Correct System Configuration:** The security and proper functioning of the system depend on the correct initial configuration of all contracts, which includes the following:
 - The `signer` address in the `SponsoredCCTPDstPeriphery`, `SponsoredOFTSrcPeriphery`, and `SponsoredCCTPSrcPeriphery` contracts must be correctly set to the public key of the trusted off-chain API.
 - The `SponsoredCCTPDstPeriphery.sol` and `DstOFTHandler.sol` contracts must be set as the owners of their corresponding `DonationBox` instance in order to withdraw sponsorship funds.
 - The mapping between EVM tokens and their HyperCore counterparts (`CoreTokenInfo`) must be correct. In particular, `CoreTokenInfo` for the `baseToken` must be properly

configured to prevent failed transactions.

- **Underlying Infrastructure Security:** The system fully inherits the security risks of its underlying infrastructure, which includes the following:
 - **Bridge Correctness:** The safety of the entire protocol rests on the correctness of the CCTP and OFT implementations, including their burn/mint mechanics, denylist functionality, fee structures, message integrity, and attestation validity.
 - **HyperEVM/HyperCore L1 Synchronization:** The system assumes that state changes made on HyperCore are reflected on HyperEVM consistently by the next block.
 - **Off-Chain Environments:** The off-chain API, signer, and permissioned bot are assumed to run in secure environments where their private keys and operational integrity are protected.
- **Nonce Management:** The nonce mechanism is assumed to be robust against replay attacks. The use of a single, application-level nonce for CCTP destination-side deduplication is trusted not to cause collisions between transactions from different source chains, which could lead to legitimate transactions being rejected.

Privileged Roles

The system is governed by several privileged roles with significant capabilities. The security of the protocol relies on the assumption that these roles will be managed securely and operated honestly. This includes protecting the private keys associated with the `Owner` and `DEFAULT_ADMIN_ROLE`.

- `DEFAULT_ADMIN_ROLE`: This is the most powerful role in the destination chain contracts (`HyperCoreFlowExecutor` and its inheritors). It has the authority to:
 - manage all other roles, including granting and revoking permissions for the `PERMISSIONED_BOT_ROLE` and `FUNDS_SWEEPER_ROLE`
 - set the trusted API `signer` address in `SponsoredCCTPDstPeriphery`
 - configure critical token parameters by calling `setCoreTokenInfo` and `setFinalTokenInfo`
 - set the authorized source periphery contracts for `DstOFTHandler`
 - set the `quoteDeadlineBuffer` in the `SponsoredCCTPDstPeriphery` contract
 - perform a powerful fund sweep from a `SwapHandler` via `sweepOnCoreFromSwapHandler`
- `Owner`: In the `SponsoredOFTSrcPeriphery` and `SponsoredCCTPSrcPeriphery` contracts, the owner (following the `Ownable` pattern) is the sole account that can set the trusted API `signer` address for the source chain.
- `PERMISSIONED_BOT_ROLE`: This role is essential for the operation of the asynchronous swap flow. It is trusted to:
 - finalize pending swaps by calling `finalizeSwapFlows`

- activate user accounts on HyperCore on behalf of the protocol via `activateUserAccount`
 - manage HyperCore limit orders by calling `submitLimitOrderFromBot` and `cancelLimitOrderByCloid`
 - send ad-hoc sponsorship funds to `SwapHandler` contracts
-
- `FUNDS_SWEEPER_ROLE` : This role is designed for fund recovery and can withdraw assets from multiple points in the system, including the `HyperCoreFlowExecutor` contract itself, and the `DonationBox` and individual `SwapHandler` contracts on both the EVM and Core layers.
 - `WHITELISTED_CALLER_ROLE` : In the `PermissionedMulticallHandler`, this role is required to execute any multicall transaction. If this permissioned handler is used by `ArbitraryEVMFlowExecutor`, the executor's address would need this role to function.
 - `SwapHandler`'s `parentHandler` : This is an architectural privilege rather than a managed role. Each `SwapHandler` contract is immutably linked to the `HyperCoreFlowExecutor` instance that deployed it. Only this parent contract can call the handler's functions, ensuring that funds within a `SwapHandler` can only be moved according to the logic of the main flow executor.

High Severity

Incorrect Additional Amount to Send in Sponsored Swaps

For sponsored swaps, the `_calcAllowableAmtsSwapFlow` function sets both the minimum and maximum acceptable output amounts to an identical, ideal value. When `isSponsored` is `true`, the calculation for `additionalToSend` is zero regardless. In the case where `limitOrderOut` is less than `maxAmountToSend`, there would not be enough tokens in `balanceRemaining`, thus, returning early with an unfinalized swap since no top-up for this is accounted for.

The impact is a denial-of-service (DoS) that critically affects the sponsored swap feature due to the fact that `limitOrderOut`, as an output swap value from the `amountInEvm` (without fees), would almost always be less than the total amount (with fees). This can cause funds to be temporarily locked in the `SwapHandler` contract until a manual administrative sweep and transfer to the correct recipient.

Consider updating the calculation for `additionalToSend` to compute the correct amount when `isSponsored` is `true`.

Update: Resolved in [pull request #36](#). For sponsored flows, `additionalToSend` is now properly calculated.

Incorrect Execution Context for Transfer on Core When Finalizing a Swap

The `HyperCoreFlowExecutor.sol` contract orchestrates a swap mechanism using distinct `SwapHandler` contracts. During a swap's initiation, funds are transferred to a `SwapHandler` instance that holds the assets on HyperCore. The `finalizeSwapFlows` function is responsible for completing the process by transferring these swapped final tokens back to the end user.

The `finalizeSwapFlows` function attempts to transfer funds by calling the `HyperCoreLib.transferERC20CoreToCore` library function directly within `_finalizeSingleSwap`. This call executes from the context of the `HyperCoreFlowExecutor` contract or its parent contract, not the `SwapHandler` contract that holds the swapped funds for the `finalToken`. Since the caller does not own the assets, the transfer operation fails and causes the transaction to revert. This results in a permanent DoS for all swap finalizations, freezing user funds in their corresponding `SwapHandler` contracts until manually recovered by a privileged administrator.

Consider modifying `_finalizeSingleSwap` to delegate the transfer call to the correct `SwapHandler`, ensuring that the transfer is executed from the context of the contract that owns the funds.

Update: Resolved in [pull request #29](#).

Incorrect Token Index Prevents Asset Rescue

The `HyperCoreFlowExecutor` contract contains a privileged `sweepOnCoreFromSwapHandler` function. This function is the sole administrative mechanism for withdrawing assets from the balance of a `SwapHandler` instance on the HyperCore layer. It is intended to be used for both routine fund management and as a critical recovery tool to rescue assets that may become trapped due to other

protocol failures.

The function incorrectly uses a market-specific `assetIndex` instead of the required token-specific `coreIndex` when instructing the `SwapHandler` to perform a transfer. This discrepancy can cause the function to transfer the wrong asset if a market `assetIndex` collides with another token's `coreIndex`. Executing this function could, therefore, lead to an unintentional sweep of an incorrect asset from the `SwapHandler`.

A more severe consequence is that other failures resulting in positive token balance in the `SwapHandler` may become irrecoverable. The `SwapHandler` contract has no other function to withdraw funds from HyperCore or bridge them back to the EVM layer. This sweep function is the only escape hatch. Since the function is not working as intended (either reverting or transferring the wrong asset), any funds that become stuck in a `SwapHandler` due to other reasons (such as regular order mismatch or bot failures) will become permanently trapped.

Consider using the correct token index (`coreTokenInfos[token].coreIndex`) in the `swapHandler.transferFundsToUserOnCore` call.

Update: Resolved in [pull request #30](#).

Partial Conversion of Initial Token Leads to Stranded Funds

The `_executeFlow` function within the `ArbitraryEVMFlowExecutor.sol` contract is responsible for orchestrating arbitrary on-chain actions, such as token swaps, via a `MulticallHandler`. This function determines the outcome of these actions by comparing the contract's token balances **before** and **after** execution. The design explicitly assumes a single-asset outcome, where the entire `amountInEVM` of the `initialToken` is either fully consumed (producing a `finalToken`) or fully refunded.

As a result of this assumption, the function relies on `balanceOf` snapshots as a binary test: if the executor's balance of the initial token is unchanged, the swap is treated as having failed. Otherwise, it is treated as successful. This only works if the arbitrary action sequence always consumes *all* of the initial token or none of it.

However, many reasonable multicall sequences may only perform a partial conversion of the `initialToken`, returning the leftover portion to the executor. In such cases, the snapshot logic interprets *any* decrease in the initial-token balance as a full conversion, and only accounts for the resulting `finalToken`. The leftover `initialToken`, which the single-output design has no place to represent, is silently ignored and becomes permanently stranded on the `ArbitraryEVMFlowExecutor` contract.

To preserve the intended single-token execution model and prevent stranded funds, the function should explicitly revert whenever a partial conversion is detected, that is, whenever the executor's post-execution balance of the `initialToken` is neither equal to its original snapshot (full refund) nor equal to `snapshot - amountInEVM` (full consumption). Enforcing this invariant restores the balance-snapshot mechanism to a safe and unambiguous binary decision and prevents users from losing funds in execution flows that violate the contract's single-output design.

Update: Acknowledged, not resolved. The Risk Labs team stated:

We do not agree with the proposed solution (i.e., to revert whenever a partial conversion is detected). This is because it could result in some transactions getting stuck as they will

always revert. We see it as the responsibility of the API to encode a set of arbitrary actions that will never leave leftover tokens behind. The `drainLeftoverTokens` function on the `multicallHandler` can also be used by the API to send leftover tokens to `finalRecipient`.

Medium Severity

Inconsistent Non-Sponsored Minimum Acceptable Output Amount

The `HyperCoreFlowExecutor` contract is designed to handle asset swaps and transfers to the HyperCore exchange. For non-sponsored swaps, the `_calcAllowableAmtsSwapFlow` function calculates the minimum and maximum acceptable output amounts.

An inconsistency exists where the minimum acceptable output (`minAllowableAmountToForwardCore`) for the non-sponsored flow can be greater than the maximum expected output. This happens when the user-set `maxUserSlippageBps` value is less than the basis point of the bridging cost. However, this situation should not be possible due to the previous [slippage check](#) when initiating a swap as the estimated slippage includes bridging fees and if it exceeds the `maxUserSlippageBps`, the transaction will revert instead. Hence, this inconsistency is not exploitable this way.

Nonetheless, consider updating the minimum or maximum acceptable output amount for the non-sponsored flow to ensure consistency and avoid any future unintended side effects.

Update: Resolved in [pull request #37](#). `maxAllowableAmountToForwardCore` is now set to the maximum amount of final tokens subtracted by the bridging fee.

Incorrect `finalToken` in `HyperEVMFallback`

The `_initiateSwapFlow` function is designed to handle token swaps. At the very beginning of this function, a check is performed to ensure that `finalRecipient` has an activated account on HyperCore. If [the account is not activated](#) and the transaction is not sponsored, the flow is diverted to `_fallbackHyperEVMFlow`. Crucially, at this point, the `params` object passed to the fallback still contains the user's intended `finalToken` (the token they wished to receive after the swap), not the `initialToken` (the token they provided for the swap).

If the contract holds any balance of the `finalToken` (e.g., due to an erroneous transfer by a user or another contract), `_fallbackHyperEVMFlow` will attempt to `safeTransfer` this `finalToken` to the caller's address. If the contract does not hold the `finalToken` or an insufficient amount, the `IERC20(params.finalToken).safeTransfer` call within `_fallbackHyperEVMFlow` will revert.

Consider modifying `params.finalToken` to `initialToken` before calling `_fallbackHyperEVMFlow` in the "check account activation block" of `_initiateSwapFlow`. Doing so ensures that the fallback mechanism performs the correct refund.

Update: Resolved in [pull request #28](#). For the HyperEVM fallback flow, the final token is now set to the initial/base token.

Stranded `baseToken` Dust in `SwapHandler` Contracts

The `HyperCoreFlowExecutor` contract facilitates token swaps by transferring user-provided `baseToken` to designated `SwapHandler` contracts for external exchange operations. These `SwapHandler` contracts execute trades on an external exchange with either buy/sell order depending on the market. The `_initiateSwapFlow` function, **transfers the entire amount** of `baseToken` to the `SwapHandler` contract.

Exchange-precision requirements can leave small amounts of `baseToken` dust within `SwapHandler` contracts after an ordinary swap. This happens due to normal transaction flow that takes into consideration the restrictions on price tick size, minimum volume size, as well as swap fees, almost always, in the input token to the swap trade (i.e., the `baseToken`). Note that, depending on the limit price, if it is a `swappmaker` instead of a `swaptaker`, one can get rebated on the `feeToken` to the spot balance after fulfillment. The decision of precise limit price or volume is computed by an off-chain bot. As such, even though it may optimize to use up the total amount of the `baseToken` from each quote, it nonetheless can almost never guarantee that for any single trade.

The `HyperCoreFlowExecutor` contract lacks a mechanism to withdraw this `baseToken` dust. The existing `sweepOnCoreFromSwapHandler` function is designed to sweep the `finalToken`, not the `baseToken`. This results in a cumulative locking up of `baseToken` for the protocol. Furthermore, dust from multiple users commingles, creating a reliance on the trusted off-chain bot to avoid misusing these funds.

Consider implementing a function within `HyperCoreFlowExecutor` that allows a privileged role to sweep `baseToken` dust from `SwapHandler` contracts. This would enable recovery of stranded assets and improve fund segregation.

Update: Resolved in [pull request #40](#). `sweepOnCoreFromSwapHandler` now sweeps both the `finalToken` and `baseToken` from the handler.

Native Tokens Are Irrecoverable

The `ArbitraryEVMFlowExecutor.sol` contract implements a `receive()` external payable function. This allows contracts that inherit it, such as `SponsoredCCTPDstPeriphery` and `DstOFTHandler`, to accept native HYPE token. This functionality appears intentional to support arbitrary cross-chain actions that may require a native token balance.

While the system provides administrative **functions to sweep various ERC-20 tokens**, it lacks an equivalent mechanism for native currency. Any native token sent to the contract address are therefore irrecoverable.

Consider implementing a guarded withdrawal function that allows a privileged address with the `FUNDS_SWEEPER_ROLE` to recover the contract's entire native token balance.

Update: Resolved in [pull request #41](#).

`HyperliquidDepositHandler` is Susceptible to Funds Griefing

The `HyperliquidDepositHandler` contract is designed to bridge tokens to Hypercore via two primary entry points: `handleV3AcrossMessage` for Across protocol fills and `depositToHypercore` for direct user deposits. Both of these paths utilize the internal `_depositToHypercore` function, which contains

logic to activate new Hypercore users by withdrawing a fee from a contract-owned `DonationBox`.

However, an attacker can repeatedly call `handleV3AcrossMessage` with a zero amount and a new user address. This action triggers the user activation logic, which unconditionally withdraws funds from the `DonationBox`, allowing for the complete draining of its balance for any supported token. The same griefing attack is possible by relayers who submit a `fillRelay` transaction to the SpokePool with a zero fill amount and a non `empty` `message` so that the `handleV3AcrossMessage` function is called to activate an arbitrary account.

Consider refactoring the account activation logic within the `HyperliquidDepositHandler` contract in order to eliminate this griefing vector. Alternatively, consider monitoring the interactions with `HyperliquidDepositHandler` to ensure rapid response to any attempted exploitation of this griefing vector.

Update: Resolved in [pull request #77](#). The Risk Labs team stated:

We have decided to address this issue by requiring the caller of the public functions to pass in a signed payload. This is so that we can have the Across API sign off on all public function calls and therefore control which accounts get activated. We preferred this approach to the `relay/tx.origin` approach for determining whether to activate an account as that is less of their concern than the APIs. 12/3 Update: We have added account-activation replay protection at commit [8dcd19a](#) due to unknown behavior of HyperLiquid's [policy](#).

Low Severity

Misleading and Non-Authoritative Whitelist Events in

`PermissionedMulticallHandler`

The `PermissionedMulticallHandler` contract implements an access-control mechanism to restrict its functionality to a set of whitelisted callers. The contract provides `whitelistCaller` and `removeCallerFromWhitelist` functions as wrappers around the standard OpenZeppelin `AccessControl` functions `grantRole` and `revokeRole`. These wrapper functions emit custom `CallerWhitelisted` and `CallerRemovedFromWhitelist` events, respectively, presumably to facilitate off-chain monitoring of the whitelist.

However, the implementation of these custom events does not accurately reflect the contract's authorization state. The `whitelistCaller` and `removeCallerFromWhitelist` functions emit events unconditionally, regardless of whether the underlying role assignment was actually changed. This can produce false positives, such as emitting `CallerWhitelisted` for an account that already has the role. Furthermore, an administrator can modify the whitelist by calling `grantRole` and `revokeRole` directly, which only emits the standard `RoleGranted` and `RoleRevoked` events. This creates false negatives, as the custom events are bypassed entirely. Off-chain systems that rely on `CallerWhitelisted` and `CallerRemovedFromWhitelist` as the source of truth will therefore maintain an incorrect state of the whitelist.

Consider making the event emissions an authoritative log of state changes. This can be achieved by ensuring that the custom events are only emitted when `WHITELISTED_CALLER_ROLE` is actually granted or revoked. One approach is to check an account's role status before calling the underlying function and emitting the event. Alternatively, the custom events could be deprecated in favor of the standard `RoleGranted` and `RoleRevoked` events, with documentation directing off-chain indexers to use these standard events as the canonical source of truth for role changes.

Update: Resolved in [pull request #56](#). The Risk Labs team stated:

We have removed `whitelistCaller(address caller)` and `removeCallerFromWhitelist(address caller)` as `grantRole` and `revokeRole` are already exposed as public functions in `AccessControl`.

Unsafe ABI Encoding

The use of `abi.encodeWithSelector` is considered unsafe. While it is not an uncommon practice to use `abi.encodeWithSelector` to generate calldata for a low-level call, it is not type-safe.

Throughout the codebase, multiple instances of `abi.encodeWithSelector` were identified:

- Within the `_buildMulticallInstructions` function in `ArbitraryEVMFlowExecutor.sol`.
- Within the `receiveMessage` and `_executeWithEVMFlow` functions in `SponsoredCCTPDstPeriphery.sol`.
- Within the `lzCompose` and `_executeWithEVMFlow` functions in `DstOFTHandler.sol`.

Consider replacing all the occurrences of unsafe ABI encodings with `abi.encodeCall` which checks whether the supplied values actually match the types expected by the called function.

Update: Resolved in [pull request #42](#).

Unchecked Return Value of `transfer` Call

The `SwapHandler` contract includes a `sweepErc20` function intended for allowing `parentHandler` to withdraw ERC-20 tokens. This function executes a `transfer` call on the specified token contract to move funds out of the handler. However, the implementation of the `sweepErc20` function does not check the boolean return value of the `transfer` call. Some non-compliant or legacy ERC-20 tokens return `false` on failure instead of reverting the transaction. If such a token is used, the `transfer` call could fail silently while the `sweepErc20` transaction itself succeeds, leading to unexpected behavior.

Consider using the `SafeERC20` library by OpenZeppelin for all ERC-20 token interactions.

Update: Resolved in [pull request #43](#).

Finalize Swap Flow Lacks Configuration Check for Final Token

The protocol uses a two-stage process for handling swaps on HyperCore. First, the `_initiateSwapFlow` function sends funds to a dedicated `SwapHandler` contract. Later, a permissioned bot calls `finalizeSwapFlows` to complete the transaction by transferring the swapped assets to the end user on HyperCore.

However, the `finalizeSwapFlows` function does not validate that a `FinalTokenInfo` struct is configured for the given `finalToken`, instead reading directly from the mapping. If the function is called with an unconfigured token, it will proceed with a `swapHandler` address of `0x0`. This causes the subsequent balance check to read from the zero address, which will return some balance and will prevent the swap from being finalized. While the function is only callable by a permissioned bot that should provide valid inputs, the contract itself does not enforce this invariant.

Consider improving the robustness of the `finalizeSwapFlows` function by adding a validation check at the start of the function. Using the `_getExistingFinalTokenInfo` helper function would ensure that `FinalTokenInfo` is properly configured for the given token before any logic is executed, preventing the function from proceeding with a null `swapHandler` and making the contract more resilient.

Update: Resolved in [pull request #44](#).

Lack of Integration and End-to-End Tests

The system is composed of multiple distinct smart contracts and off-chain components that interact to facilitate complex, asynchronous cross-chain flows. These components include source and destination periphery contracts for both CCTP and OFT, core logic executors, and external dependencies such as the HyperCore L1 precompiles and a trusted off-chain bot. The correctness of the system relies on the seamless and predictable interaction between all of these parts.

The current test suite does not sufficiently cover the integration points across these components. While unit tests may verify the logic of individual contracts, there is an absence of integration or end-to-end tests that simulate a complete transaction lifecycle. This makes it difficult to validate critical and complex interactions, such as the two-stage asynchronous swap flow involving the off-chain bot or the precise behavior of calls between HyperEVM and HyperCore. Without a comprehensive test suite, there is a higher risk that subtle bugs, incorrect assumptions about external dependencies, or future regressions could go undetected.

Consider implementing a dedicated suite of integration and end-to-end tests to provide stronger assurances about the system's overall correctness. This suite should cover the full lifecycle of a transaction for all supported execution modes, from the source periphery contract to the final settlement on the destination. It should also include tests for failure scenarios, such as fallback flows, and simulate the behavior of external components like the off-chain bot and HyperCore precompiles. A robust testing framework would significantly improve the long-term security and maintainability of the protocol.

Update: Acknowledged, will resolve. The Risk Labs team stated:

We acknowledge that there should be more end-to-end tests. We will be writing more tests after this audit is completed.

Sponsorship Logic Fully Cancels on Partial Insufficiency

The `HyperCoreFlowExecutor` contract contains the `_executeSimpleTransferFlow` function that orchestrates the deposit of tokens into a user's HyperCore account. This flow includes a mechanism to sponsor user bridging fees, where the `amountToSponsor` is calculated based on the user's quote. Before executing the transfer, the logic `validates` whether the required sponsorship funds are available in the `donationBox` contract.

However, the current implementation of the fund availability check is binary. If the `donationBox` contains a balance that `is less than` the fully calculated `amountToSponsor`, the sponsorship is cancelled entirely by `setting amountToSponsor to zero`. This behavior is suboptimal, as it fails to utilize available funds if they are not sufficient to cover the entire fee. Consequently, users receive no sponsorship even when some funds were available, and residual token balances can become stranded in the `donationBox`, leading to inefficient use of the protocol's sponsorship capital.

Consider refining the sponsorship logic to allow for partial sponsorship. Instead of resetting `amountToSponsor` to zero when the `donationBox` balance is insufficient for the full amount, the logic should set it to the lesser of the calculated sponsorship fee and the available balance in the `donationBox`. This change would ensure that all available funds are used as effectively as possible, maximizing the benefit to users and preventing token dust from accumulating in the sponsorship contract.

Update: Resolved in [pull request #45](#).

Application Payloads Lack Versioning

The system transmits application-specific parameters from the source to the destination chain using custom-encoded data payloads. In the OFT flow, this payload is the `composeMsg`, while in the CCTP flow, it is the `hookData`. Both payloads are encoded with a fixed data structure, which the destination handlers rely on for decoding and execution.

However, neither the `composeMsg` nor the `hookData` payload includes a version number or discriminator. The decoding logic on the destination chain is, therefore, tightly coupled with a single, rigid data structure. This design is not forward-compatible and creates a significant risk during future upgrades. If the structure of either payload is changed on a source contract before the corresponding destination contract is updated, the destination handler will receive a message it cannot parse correctly. This will likely cause the transaction to revert, stranding the user's funds in the destination contract until they can be manually recovered.

Consider introducing a versioning system for these application-level payloads to make future upgrades more robust and resilient against deployment errors. This can be achieved by prepending a version byte to the encoded `composeMsg` and `hookData`. The destination handlers could then inspect this version upon receipt, explicitly reject messages with unknown future versions, and optionally maintain backward compatibility during a transition period. This would prevent parsing errors and ensure that funds do not become stranded due to staggered or unsynchronized deployments across chains.

Update: Acknowledged, not resolved. The Risk Labs team stated:

We acknowledge that having a version number could protect against misconfigured upgrades and mismatches of data structs. However, in the initial version of this system,

there is no upgradeability, and it is intended to keep data structures as simple/concise as possible. So, we will acknowledge this issue with no change.

Unnecessary payable Fallback Function

The `BaseModuleHandler` contract acts as a proxy, forwarding calls to an underlying `HyperCoreFlowExecutor` module via a `delegatecall` in its `fallback` function. This design allows the handler to execute logic defined in the `HyperCoreFlowExecutor` contract within its own context. The `fallback` function within the `BaseModuleHandler` contract is marked `payable`, which allows it to receive native assets. However, the logic executed through the `delegatecall` resides in the `HyperCoreFlowExecutor` contract, which does not contain any `payable` functions. Consequently, there is no functionality that requires the `BaseModuleHandler` to receive native assets through its fallback mechanism, making the `payable` modifier unnecessary.

To align the contract's implementation with its actual behavior and reduce its attack surface, consider removing the `payable` modifier from the `fallback` function in the `BaseModuleHandler` contract.

Update: Resolved in [pull request #65](#).

Inconsistent Storage Layout Pattern

The contracts employ a hybrid storage architecture to manage state. This approach combines Solidity's default state variable layout with a `namespaced storage pattern`, where a struct is explicitly assigned to a specific storage slot. This design was chosen to address contract size limits while minimizing code changes. While this hybrid model is functional and no storage collisions were identified, it introduces complexity and deviates from a uniform state management strategy. Maintaining two different storage patterns within the same contract system increases the cognitive load for developers and auditors, making the code harder to reason about and maintain.

To enhance code clarity and long-term maintainability, consider adopting a single, consistent storage pattern. Consider refactoring the contracts to exclusively use a namespaced storage layout. Doing so would create a unified and more predictable state architecture, simplifying future development and reducing the potential for storage-related issues.

Update: Resolved in [pull request #66](#) and [pull request #73](#).

Lack of Access Control Can Lead to Loss of Contract-Held Funds

The `HyperliquidDepositHandler` contract includes the `handleV3AcrossMessage` function, which serves as an entry point for the SpokePool. The intended flow is for the SpokePool to transfer tokens to the handler contract and then immediately call this function to bridge those tokens to a specified user on Hypercore.

The `handleV3AcrossMessage` function is `external` and lacks any access control, permitting any address to call it. Although the contract is not designed to hold a token balance, any funds present within it are vulnerable to theft. An attacker can invoke this function, passing an amount equal to the contract's balance of a supported token. The function will then proceed to `bridge` these funds to a Hypercore user designated by the attacker, effectively draining the contract of those assets. Such a

scenario could arise if tokens are mistakenly transferred to the contract or remain after an incomplete transaction.

Consider adding an explicit warning within the contract's source code to alert developers to this risk. A NatSpec comment, [similar to the one present in the MulticallHandler](#), should clarify that the contract is not intended to hold funds and that any tokens transferred to it can be retrieved by an arbitrary caller. This would ensure that developers interacting with the contract are fully aware of its behavior and the potential for loss of funds.

Update: Resolved in [pull request #77](#) at commit [51adcf7](#).

Lack of Event Emission

The [HyperliquidDepositHandler](#) contract includes an [addSupportedToken](#) function, which is restricted to the contract owner. This function is responsible for configuring new tokens that the handler will support, by setting their EVM address, corresponding Hypercore token ID, activation fee, and decimal difference within the [supportedTokens](#) mapping.

However, this state-changing operation does not emit an event. The absence of an event makes it challenging for off-chain systems, such as block explorers, monitoring tools, or front-end applications, to track when new tokens are added, verify the parameters used, or reconstruct the historical configuration of supported tokens. This reduces transparency and auditability of the contract's administrative actions.

Consider emitting an event whenever the [addSupportedToken](#) function is successfully executed.

Update: Resolved in [pull request #77](#).

Notes & Additional Information

Unused Variables

Throughout the codebase, multiple instances of unused variables were identified:

- In [ArbitraryEVMFlowExecutor.sol](#), the [BPS_DECIMALS](#) state variable
- In [HyperCoreFlowExecutor.sol](#), the [PX_D](#) state variable
- In the [setFinalTokenInfo](#) function, the [accountActivationFeeToken](#) variable.

To improve the overall clarity and intent of the codebase, consider removing any unused variables.

Update: Resolved in [pull request #46](#).

Unused Errors

Throughout the codebase, multiple instances of unused errors were identified:

- The [TransferAmtExceedsAssetBridgeBalance](#) error in [HyperCoreLib.sol](#)

- The `InsufficientFinalBalance` error in `ArbitraryEVMFlowExecutor.sol`
- The `DonationBoxInsufficientFundsError` error in `HyperCoreFlowExecutor.sol`

To improve the overall clarity, intentionality, and readability of the codebase, consider either using or removing any currently unused errors.

Update: Resolved in [pull request #47](#).

Unused Imports

`import { SendParam, MessagingFee } from "../../interfaces/IOFT.sol";` in `Structs.sol` is unused and could be removed.

Consider removing unused imports to improve the overall clarity and readability of the codebase.

Update: Resolved in [pull request #48](#).

Prefix Increment Operator (`++i`) Can Save Gas in Loops

The `loop counter increment` in `MulticallHandler.sol` could be more efficient.

Consider using the prefix increment operator (`++i`) instead of the postfix increment operator (`i++`) in order to save gas. This optimization skips storing the value before the incremental operation, as the return value of the expression is ignored.

Update: Resolved in [pull request #49](#).

Multiple Optimizable State Reads

Throughout the codebase, the following instance of optimizable storage reads was identified:

- The `coreTokenInfo` struct is read multiple times from storage in the `_executeSimpleTransferFlow` function.

Consider reducing SLOAD operations that consume unnecessary amounts of gas by caching the values in a memory variable.

Update: Resolved in [pull request #50](#).

Incomplete Docstrings

Throughout the codebase, multiple instances of incomplete docstrings were identified:

- In `MulticallHandler.sol`, the `handleV3AcrossMessage` function has an undocumented `token` parameter.
- In `PermissionedMulticallHandler.sol`, the `CallerWhitelisted` event has an undocumented `caller` parameter.

- In `PermissionedMulticallHandler.sol`, the `CallerRemovedFromWhitelist` event has an undocumented `caller` parameter.
- In `ArbitraryEVMFlowExecutor.sol`, the `ArbitraryActionsExecuted` event has undocumented `quoteNonce`, `initialToken`, `initialAmount`, `finalToken`, and `finalAmount` parameters.
- In `HyperCoreFlowExecutor.sol`, the `DonationBoxInsufficientFunds` event has undocumented `quoteNonce`, `token`, `amount`, and `balance` parameters.
- In `HyperCoreFlowExecutor.sol`, the `AccountNotActivated` event has undocumented `quoteNonce` and `user` parameters.
- In `HyperCoreFlowExecutor.sol`, the `SimpleTransferFlowCompleted` event has undocumented `quoteNonce`, `finalRecipient`, `finalToken`, `evmAmountIn`, `bridgingFeesIncurred`, and `evmAmountSponsored` parameters.
- In `HyperCoreFlowExecutor.sol`, the `FallbackHyperEVMFlowCompleted` event has undocumented `quoteNonce`, `finalRecipient`, `finalToken`, `evmAmountIn`, `bridgingFeesIncurred`, and `evmAmountSponsored` parameters.
- In `HyperCoreFlowExecutor.sol`, the `SwapFlowInitialized` event has undocumented `quoteNonce`, `finalRecipient`, `finalToken`, `evmAmountIn`, `bridgingFeesIncurred`, `coreAmountIn`, `minAmountToSend`, and `maxAmountToSend` parameters.
- In `HyperCoreFlowExecutor.sol`, the `SwapFlowFinalized` event has undocumented `quoteNonce`, `finalRecipient`, `finalToken`, `totalSent`, and `evmAmountSponsored` parameters.
- In `HyperCoreFlowExecutor.sol`, the `CancelledLimitOrder` event has undocumented `token` and `cloid` parameters.
- In `HyperCoreFlowExecutor.sol`, the `SubmittedLimitOrder` event has undocumented `token`, `priceX1e8`, `sizeX1e8`, and `cloid` parameters.
- In `HyperCoreFlowExecutor.sol`, the `SwapFlowTooExpensive` event has undocumented `quoteNonce`, `finalToken`, `estBpsSlippage`, and `maxAllowableBpsSlippage` parameters.
- In `HyperCoreFlowExecutor.sol`, the `UnsafeToBridge` event has undocumented `quoteNonce`, `token`, and `amount` parameters.
- In `HyperCoreFlowExecutor.sol`, the `SponsoredAccountActivation` event has undocumented `quoteNonce`, `finalRecipient`, `fundingToken`, and `evmAmountSponsored` parameters.
- In `HyperCoreFlowExecutor.sol`, the `SetCoreTokenInfo` event has undocumented `token`, `coreIndex`, `canBeUsedForAccountActivation`, `accountActivationFeeCore`, and `bridgeSafetyBufferCore` parameters.
- In `HyperCoreFlowExecutor.sol`, the `SentSponsorshipFundsToSwapHandler` event has undocumented `token` and `evmAmountSponsored` parameters.
- In `HyperCoreFlowExecutor.sol`, the `setCoreTokenInfo` function has undocumented `token`, `coreIndex`, `canBeUsedForAccountActivation`, `accountActivationFeeCore`, and `bridgeSafetyBufferCore` parameters.

- In `HyperCoreFlowExecutor.sol`, the `predictSwapHandler` function has an undocumented `finalToken` parameter, and not all return values are documented.
- In `HyperCoreFlowExecutor.sol`, the `finalizeSwapFlows` function has undocumented `finalToken`, `quoteNonces`, and `limitOrderOuts` parameters, and not all return values are documented.
- In `HyperCoreFlowExecutor.sol`, the `cancelLimitOrderByCloid` function has undocumented `finalToken` and `clويد` parameters.
- In `HyperCoreFlowExecutor.sol`, the `sendSponsorshipFundsToSwapHandler` function has an undocumented `amount` parameter.
- In `DstOFTHandler.sol`, the `SetAuthorizedPeriphery` event has undocumented `srcEid` and `srcPeriphery` parameters.
- In `DstOFTHandler.sol`, the `lzCompose` function has an undocumented `_message` parameter.
- In `SponsoredOFTSrcPeriphery.sol`, the `SponsoredOFTSend` event has undocumented `quoteNonce`, `originSender`, `finalRecipient`, `destinationHandler`, `quoteDeadline`, `maxBpsToSponsor`, `maxUserSlippageBps`, `finalToken`, and `sig` parameters.
- In `SponsoredOFTSrcPeriphery.sol`, the `deposit` function has undocumented `quote` and `signature` parameters.

Consider thoroughly documenting all functions/events (and their parameters or return values) that are part of a contract's public API. When writing docstrings, consider following the [Ethereum Natural Specification Format](#) (NatSpec).

Update: Resolved in [pull request #58](#).

Boolean Comparisons Reduce Readability

Throughout the codebase, multiple instances of conditional checks were identified that compare boolean values against the literal `false` instead of using the more logical NOT operator (`!`). While functionally correct, this pattern makes the code less concise and reduces readability for developers accustomed to standard Solidity conventions.

- Boolean literal `False` within the contract `HyperCoreFlowExecutor` in `HyperCoreFlowExecutor.sol`
- Boolean literal `False` within the contract `DstOFTHandler` in `DstOFTHandler.sol`

Consider refactoring these comparisons to use the logical NOT operator (`!`). Doing so help will improve code clarity and align the codebase with common best practices, making the logic more intuitive and maintainable.

Update: Resolved in [pull request #51](#).

Typographical Errors

Throughout the codebase, multiple instances of typographical errors were identified:

- 'Calcuates' instead of 'Calculates'
- 'form' instead of 'from'
- 'revover' instead of 'recover'.
- 'receipent' instead of 'recipient'.

To improve code readability, consider correcting any typographical errors in the codebase.

Update: Resolved in [pull request #52](#) and [pull request #78](#).

Misleading Documentation

This statement is not entirely correct. If the mint recipient is not this contract, the funds will not be minted to this `SponsoredCCTPDstPeriphery` contract. Instead, the funds will be **minted** to the `mintRecipient` in the **call to** `receiveMessage` and, therefore, will not be kept in `SponsoredCCTPDstPeriphery` contract.

Consider correcting the aforementioned comment to improve the overall clarity and readability of the codebase.

Update: Resolved in [pull request #53](#).

Unaddressed TODO Comments

During development, having well-described TODO comments will make the process of tracking and resolving them easier. However, these comments might age and important information for the security of the system might be forgotten by the time it is released to production.

Throughout the codebase, multiple instances of unaddressed TODO comments were identified:

- In [line 59](#) of `SponsoredCCTPInterface`
- In [line 48](#) of `HyperCoreLib.sol`

Consider removing all instances of TODO comments and instead tracking them in the issues backlog. Alternatively, consider linking each inline TODO to a corresponding backlog issue.

Update: Resolved in [pull request #54](#).

Lack of Documentation

The `HyperCoreFlowExecutor` contract provides core logic that is designed to be executed within the context of the two periphery contracts (`SponsoredCCTPDstPeriphery` and `DstOFTHandler`). These contracts utilize `delegatecall` to access and run the functions defined in `HyperCoreFlowExecutor` , allowing for code reuse while maintaining separate storage for each handler.

However, contract's documentation does not explicitly state that it is intended to be used as a `delegatecall` target and that it should not be interacted with directly. This absence of guidance may lead to incorrect usage by developers. For instance, functions like `predictSwapHandler` rely on the caller's address (`address(this)`) for their calculations. If this function is called directly on the `HyperCoreFlowExecutor` contract, it will produce an incorrect result based on its own address rather than the address of the intended handler contract.

To prevent misuse and ensure the system's architectural integrity, consider adding prominent documentation to the `HyperCoreFlowExecutor` contract. This documentation should clarify that it is an implementation contract that is intended to be used via `delegatecall` and that direct calls may lead to unexpected behavior.

Update: Resolved in [pull request #64](#).

Unoptimized USDC Transfer to HyperCore

The `HyperCoreFlowExecutor` contract is designed to facilitate asset transfers from HyperEVM to HyperCore. When processing these transfers for end users, it relies on the `HyperCoreLib` library. For USDC transactions, this library interacts with the `ICoreDepositWallet` contract to bridge the funds and credit the designated recipient's account on HyperCore.

The `transferERC20EVMToCore` function within `HyperCoreLib` currently handles USDC transfers through a two-step process. It first calls the `deposit` function of the `ICoreDepositWallet` contract, which credits the `HyperCoreFlowExecutor` contract's account on HyperCore. Subsequently, it executes `transferERC20CoreToCore`, which makes a `SPOT_SEND` precompile call to transfer the funds from the contract's account to the final recipient. While `transferERC20CoreToCore` is necessary for other tokens, for USDC, this sequence involves two separate state-changing operations on HyperCore where a single operation could suffice.

Consider optimizing the logic within the `HyperCoreLib` library by using the `depositFor` function available in the `ICoreDepositWallet` contract for USDC transfers. This function allows for specifying the final recipient in a single transaction, which would credit the end-user's account directly. Adopting this approach would consolidate the two state-changing operations into one for USDC, potentially reducing transaction costs and gas consumption for these specific transfers.

Update: Acknowledged, not resolved. The team stated:

This change would only affect USDC flows, and all of the other flows would remain the same. Adding this change would introduce quite substantial diff for a minimal impact. We prefer to keep it as is so as to not extend the project launch timeline.

Client Reported

Not Forwarding The Exact OFT Messaging Fees

The `SponsoredOFTSrcPeriphery` contract facilitates cross-chain transactions by wrapping a

LayerZero OFT (Omnichain Fungible Token) message. The contract's payable `deposit` function is intended to accept native token from a user to pay for the cross-chain messaging fee. It calculates the required fee and then calls the `send` function forwarding `msg.value` instead of the exact quoted fee on the underlying `OFT_MESSENGER`.

The `deposit` function forwards the entire `msg.value` to the messenger contract without performing any validation. The underlying messenger, which inherits its logic from LayerZero's `OAppSender`, may enforce a strict equality check between the `msg.value` and the required native fee, as seen in the `_payNative` function of some implementations. If a user provides any amount of native token that is not exactly equal to the required fee, the transaction will revert from the underlying contract. This creates a significant usability issue, effectively leading to a denial of service for users who do not calculate the fee perfectly.

Consider rejecting a transaction early and with a clear error message when `msg.value < fee.nativeFee`. In cases where `msg.value > fee.nativeFee`, consider forwarding the exact `fee.nativeFee` to the `send` function and refunding the difference to the user.

Update: Resolved in [pull request #22](#).

Incorrect Spot Price Decimal Conversion

The `_getApproxRealizedPrice` function of the `HyperCoreFlowExecutor` contract incorrectly assumes that the spot price fetched from the Hyperliquid Core (`HyperCoreLib.spotPx`) has a fixed precision of 8 decimals. According to the [documentation](#), the raw spot price is a scaled integer, and its human-readable floating point value is `raw_price / 10^(8 - base asset szDecimals)`, where `szDecimals` is a property of the base asset of the spot market. By failing to account for the asset's specific `szDecimals`, the contract misinterprets the price, causing all subsequent price and slippage calculations to become incorrect.

Consider correctly computing the raw spot price decimal with respect to the base asset `szDecimals` of the relevant spot market.

Update: Resolved in [pull request #32](#).

Incorrect Market Index for Limit Orders

When submitting a limit order, the market index is incorrectly set to the asset index. According to the intended logic for limit orders, the market index should be calculated as `10000 + asset index`. This affects both `submitLimitOrderFromBot` and `cancelLimitOrderByCloid`.

Consider correctly setting the market index to `10000 + asset index` when submitting a limit order.

Update: Resolved in [pull request #33](#).

Incorrect Handling Of The Sponsored Account Activation Fee

The `activateUserAccount` function is designed to sponsor the activation of a user's account on HyperCore. However, the current implementation incorrectly [transfers the activation fee directly](#) to the `finalRecipient`'s address on HyperCore.

Consider correctly handling the activation fee by having the sponsoring contract fund its own

HyperCore account and then perform an action (like a 1 wei transfer) that triggers the recipient's account activation, at which point the fee is deducted from the sponsoring contract's balance by the system.

Update: Resolved in [pull request #31](#).

Conclusion

The audited codebase introduces a sophisticated framework for sponsored, cross-chain transactions. The system utilizes LayerZero's OFT and Circle's CCTP as underlying bridges to facilitate complex flows, including token swaps on HyperCore and the execution of arbitrary actions on HyperEVM. The architecture relies on a trusted off-chain entity to authorize transactions by providing signed quotes, which define the parameters and execution mode for each cross-chain operation.

The audit identified several areas for improvement, including four high-severity issues. These findings primarily relate to issues in the execution context, token configuration, and internal accounting logic, which, in certain scenarios, could lead to incorrect fund transfers or stranded assets. Given the complexity of the protocol and its reliance on asynchronous, multi-component interactions, the codebase would significantly benefit from a comprehensive integration test suite. Many of the medium- and high-severity issues identified during this engagement could have been detected during development with more robust end-to-end testing.

The Risk Labs team is appreciated for being highly responsive and providing valuable insights throughout the engagement. Their commitment to addressing the findings and improving the security posture of the protocol is commendable.

Ready to secure your code?

[Request an Audit→](#)

Smart Contracts

Contracts Libraries

Contracts Wizard ↗

Contracts MCP ↗

Upgrades Plugin ↗

Open Source Tools

Relayer

Monitor

UI Builder ↗

Safe Utils ↗

Role Manager ↗

The Open Source Stack

Blockchains

Arbitrum

Base

Ethereum

Linea

Midnight

OP Mainnet

Polkadot

Starknet

Stellar

Zama

ZKsync

Company

About Us

Why OpenZeppelin

News

Customer Stories

Careers

Brand Kit

Trust ↗

Terms & Policies

Terms of Service

Privacy Policy

Trademark Guidelines

AGPL License

Services

Smart Contract Security Audit

Blockchain Infrastructure Audit

ZKP Audit

Security Services

Solutions

Ecosystem Stack

Financial Institutions Stack



© 2026 Zeppelin Group Ltd

Resources

Research

Documentation ↗

Stats

Forum ↗

Ethernaut CTF ↗